

SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Akileshwaran A
192511354RD

CO3-AT1-Debugging and Optimization

[← Back](#)

QUESTIONS

Q1ATM Withdrawal – Exception
Handling Debugging
10 marks**Q2**Hospital Registration –
NullPointerException
Debugging
10 marks**Q3**E-Commerce Product Search –
Array Exception Debugging
10 marks**Q4**Railway Reservation –
Synchronization Debugging
10 marks**Q5**Bank Account – Race Condition
Debugging

Q4 Railway Reservation – Synchronization Debugging

10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {  
  
    int seats = 1;  
  
    void bookSeat(String customer) {  
  
        if (seats > 0) {
```

10 marks

5/5 answered

```
System.out.println(customer
+ " is booking a seat");

try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}

seats--;

System.out.println(customer
+ " booking successful");

} else {
    System.out.println(customer
+ " - No seat available");
}
}

class Customer extends Thread {

    Reservation reservation;
    String customerName;

    Customer(Reservation reservation, String customerName) {
        this.reservation = reservation;
        this.customerName = customerName;
    }
}
```

```
public void run() {  
    reservation.bookSeat(customerName);  
}  
}  
  
class RailwayReservation {  
  
    public static void main(String[] args) {  
  
        Reservation reservation = new Reservation();  
  
        Customer c1 =  
            new Customer(reservation, "Customer 1");  
  
        Customer c2 =  
            new Customer(reservation, "Customer 2");  
  
        c1.start();  
        c2.start();  
    }  
}
```

The corrected program must protect the shared seat resource using synchronization.

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.*;  
2 public class Main{  
3     public static void main (String args[]){  
4         reservation r= new reservation();
```

```
5         customer c1=new customer(r,"Customer 1");
6         customer c2=new customer(r,"Customer 2");
7         c1.start();
8         c2.start();
9     }
10 }
11 class reservation{
12     int s=1;
13     synchronized void bookseat(String customer){
14         if(s>0){
15             System.out.println(customer+" is booking a seat");
16
17             try{
18                 Thread.sleep(100);
19             }
20             catch(InterruptedException e){
21                 System.out.println("thread interrupted");
22             }
23             s--;
24             System.out.println(customer+" Booking successful");
25         }
26     else
27         System.out.println(customer+"- no seat available");
28     }
29 }
30 class customer extends Thread{
31     reservation r;
32     String customername;
33     customer(reservation r,String customername){
34         this.r=r;
35         this.customername=customername;
36     }
37     public void run(){
38         r.bookseat(customername);
```

```
39      }  
40  }
```

Test Input

stdin input (optional)

Output

```
Customer 1 is booking a seat  
Customer 1 Booking succcessful  
Customer 2- no seat available
```

🕒 Pending manual review

✅ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.